Business Case Study - Twitter NER NLP

Problem Statement

Twitter is a microblogging and social networking service on which users post and interact with messages known as "tweets". Every second, on average, around 6,000 tweets are tweeted on Twitter, corresponding to over 350,000 tweets sent per minute, 500 million tweets per day.

Twitter wants to automatically tag and analyze tweets for better understanding of the trends and topics without being dependent on the hashtags that the users use. Many users do not use hashtags or sometimes use wrong or mis-spelled tags, so they want to completely remove this problem and create a system of recognizing important content of the tweets.

Named Entity Recognition (NER) is an important subtask of information extraction that seeks to locate and recognise named entities.

You need to train models that will be able to identify the various named entities

Data Description

Dataset is annotated with 10 fine-grained NER categories: person, geo-location, company, facility, product,music artist, movie, sports team, tv show and other. Dataset was extracted from tweets and is structured in CoNLL format., in English language. Containing in Text file format.

The CoNLL format is a text file with one word per line with sentences separated by an empty line. The first word in a line should be the word and the last word should be the label.

Consider the two sentences below;

Harry Potter was a student living in london

Albus Dumbledore went to the Disney World

These two sentences can be prepared in a CoNLL formatted text file as follows.

- Harry B-PER
- Potter I-PER
- was O
- a O
- student O
- Living O
- in O
- London B-geo-loc
- Albus B-PER
- Dumbledore I-PER
- went O
- to O
- the O
- Disney B-facility

- World I-facility

In [34]:
```
# Import the necessary libraries

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import re
import os

import warnings
warnings.filterwarnings('ignore')
```

1. Defining problem statement, importing the data and data structure analysis

1. Loading and formatting the data

Problem Statement:

Twitter wants to automatically identify and tag important entities in tweets without relying on user-provided hashtags. The goal is to build a Named Entity Recognition (NER) system that can classify each token in a tweet into one of the predefined entity categories such as person, company, geo-location, product, etc., using annotated historical tweet data.

In [35]:
```
path = r'/content/wnut 16.txt.conll'

df_txt = pd.read_csv(
    path,
    sep=r"\s+",              # split on whitespace (space or tab)
    header=None,             # no header row in file
    names=["token", "label"],# adjust if you have more columns
    comment="#",             # skip comment/meta lines starting with '#'
    skip_blank_lines=True,   # ignore sentence-separating blank lines
    engine="python",         # more flexible parser for variable whitespace
)

print(df_txt.head())
print(df_txt.tail())
```

```
            token label
0  @SammieLynnsMom     O
1         @tg10781     O
2             they     O
3             will     O
4               be     O
            token label
46013     whatchu     O
46014         got     O
46015         for     O
46016          me     O
46017  @kanyewest     O
```

In [36]:
```
path = r"/content/wnut 16test.txt.conll"

df_test = pd.read_csv(
    path,
    sep=r"\s+",              # split on whitespace (space or tab)
    header=None,             # no header row in file
    names=["token", "label"],# adjust if you have more columns
    comment="#",             # skip comment/meta lines starting with '#'
    skip_blank_lines=True,   # ignore sentence-separating blank lines
    engine="python",         # more flexible parser for variable whitespace
)

print(df_txt.head())
print(df_txt.tail())
```

```
             token label
0  @SammieLynnsMom     O
1         @tg10781     O
2             they     O
3             will     O
4               be     O
            token label
46013     whatchu     O
46014         got     O
46015         for     O
46016          me     O
46017  @kanyewest     O
```

In [37]:
```
print(df_txt.shape)
print(df_txt['label'].unique())
```

```
(46018, 2)
['O' 'B-geo-loc' 'B-facility' 'I-facility' 'B-movie' 'I-movie' 'B-company'
 'B-product' 'B-person' 'B-other' 'I-other' 'B-sportsteam' 'I-sportsteam'
 'I-product' 'I-company' 'I-person' 'I-geo-loc' 'B-tvshow' 'B-musicartist'
 'I-musicartist' 'I-tvshow' None]
```

In [38]:
```
print(df_test.shape)
print(df_test['label'].unique())
```

```
(59938, 2)
['B-other' 'I-other' 'O' 'B-movie' 'B-person' 'I-person' 'B-geo-loc'
 'B-company' 'I-company' 'B-product' 'I-product' 'B-musicartist'
 'I-musicartist' 'B-sportsteam' 'B-facility' 'I-facility' 'I-geo-loc' None
 'I-movie' 'I-sportsteam' 'B-tvshow' 'I-tvshow']
```

In [39]:
```
df_txt['label'].value_counts()
```

Out[39]:

label	count
O	43557
B-person	449
I-other	320
B-geo-loc	274
B-other	224
I-person	215
B-company	170
I-facility	105
B-facility	104
B-product	97
I-product	80
I-musicartist	61
B-musicartist	55
B-sportsteam	49
I-geo-loc	49
I-movie	46
I-company	36
B-movie	34
B-tvshow	31
I-tvshow	30
I-sportsteam	23

dtype: int64

In [40]:
```
df_test['label'].value_counts()
```

Out[40]:

label	count
O	54269
B-geo-loc	764
B-company	585
I-other	553
B-other	532
I-product	498
B-person	466
I-facility	366
I-person	300
I-company	265
B-facility	241
I-geo-loc	218
B-product	213
B-musicartist	180
I-musicartist	140
B-sportsteam	128
I-movie	48
I-sportsteam	48
I-tvshow	40
B-movie	28
B-tvshow	24

dtype: int64

In [41]:
```
df_txt.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 46018 entries, 0 to 46017
Data columns (total 2 columns):
 #   Column  Non-Null Count  Dtype 
---  ------  --------------  ----- 
 0   token   46018 non-null  object
 1   label   46009 non-null  object
dtypes: object(2)
memory usage: 719.2+ KB
```

In [42]:
```
df_test.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 59938 entries, 0 to 59937
Data columns (total 2 columns):
 #   Column  Non-Null Count  Dtype 
---  ------  --------------  ----- 
 0   token   59910 non-null  object
 1   label   59906 non-null  object
dtypes: object(2)
memory usage: 936.7+ KB
```

2. Getting the correct model and tokenizer

In [43]:
```
df_test.isnull().sum()
```

Out[43]:

	0
token	28
label	32

dtype: int64

In [44]:
```
df_txt.isnull().sum()
```

Out[44]:

	0
token	0
label	9

dtype: int64

In [45]:
```
# Drop rows where label is None / NaN
df_txt = df_txt.dropna(subset=["label"]).reset_index(drop=True)
```

In [46]:
```
# Get all unique labels from training data
unique_labels = sorted(df_txt["label"].unique())

print("Unique Labels:")
print(unique_labels)

print("\nNumber of labels:", len(unique_labels))
```

```
Unique Labels:
['B-company', 'B-facility', 'B-geo-loc', 'B-movie', 'B-musicartist', 'B-other', 'B-person', 'B-product',
'B-sportsteam', 'B-tvshow', 'I-company', 'I-facility', 'I-geo-loc', 'I-movie', 'I-musicartist', 'I-othe
r', 'I-person', 'I-product', 'I-sportsteam', 'I-tvshow', 'O']

Number of labels: 21
```

In [47]:
```
label2id = {label: idx for idx, label in enumerate(unique_labels)}
id2label = {idx: label for label, idx in label2id.items()}

print("\nLabel to ID mapping:")
print(label2id)
```

```
Label to ID mapping:
{'B-company': 0, 'B-facility': 1, 'B-geo-loc': 2, 'B-movie': 3, 'B-musicartist': 4, 'B-other': 5, 'B-pers
on': 6, 'B-product': 7, 'B-sportsteam': 8, 'B-tvshow': 9, 'I-company': 10, 'I-facility': 11, 'I-geo-loc':
12, 'I-movie': 13, 'I-musicartist': 14, 'I-other': 15, 'I-person': 16, 'I-product': 17, 'I-sportsteam': 1
8, 'I-tvshow': 19, 'O': 20}
```

In [48]:
```
from transformers import AutoTokenizer, AutoModelForTokenClassification

model_name = "bert-base-cased"

tokenizer = AutoTokenizer.from_pretrained(model_name)

model = AutoModelForTokenClassification.from_pretrained(
    model_name,
    num_labels=len(unique_labels),
    label2id=label2id,
    id2label=id2label,
)
```

```
Some weights of BertForTokenClassification were not initialized from the model checkpoint at bert-base-ca
sed and are newly initialized: ['classifier.bias', 'classifier.weight']
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and infer
ence.
```

In [49]:
```
def load_conll(path):
    sentences = []
    sentence = []

    with open(path, "r", encoding="utf-8") as f:
        for line in f:
            line = line.strip()
```

```
            # sentence boundary or comment
            if not line or line.startswith("#"):
                if sentence:
                    sentences.append(sentence)
                    sentence = []
                continue

            parts = line.split()
            token, label = parts[0], parts[-1]
            sentence.append((token, label))

    # add last sentence if file doesn't end with blank line
    if sentence:
        sentences.append(sentence)

    return sentences

train_path = "/content/wnut 16.txt.conll"
test_path  = "/content/wnut 16test.txt.conll"

train_sentences = load_conll(train_path)
test_sentences  = load_conll(test_path)

print("Number of train sentences:", len(train_sentences))
print("First train sentence sample:", train_sentences[0][:10])
```

```
Number of train sentences: 2578
First train sentence sample: [('@SammieLynnsMom', 'O'), ('@tg10781', 'O'), ('they', 'O'), ('will', 'O'),
('be', 'O'), ('all', 'O'), ('done', 'O'), ('by', 'O'), ('Sunday', 'O'), ('trust', 'O')]
```

In [50]:

```
train_tokens = [[tok for tok, lab in sent] for sent in train_sentences]
train_labels = [[lab for tok, lab in sent] for sent in train_sentences]

test_tokens = [[tok for tok, lab in sent] for sent in test_sentences]
test_labels = [[lab for tok, lab in sent] for sent in test_sentences]
```

In [51]:

```
import torch

def tokenize_and_align_labels(texts, labels, tokenizer, label2id, max_length=128):
    # texts is a list of list-of-tokens
    encodings = tokenizer(
        texts,
        is_split_into_words=True,
        truncation=True,
        padding=True,
        max_length=max_length,
        return_tensors="pt"
    )

    all_label_ids = []

    for i, sentence_labels in enumerate(labels):
        word_ids = encodings.word_ids(batch_index=i)  # map tokens → word index
        label_ids = []
        previous_word_idx = None

        for word_idx in word_ids:
            if word_idx is None:
                # Special tokens ([CLS], [SEP], [PAD])
                label_ids.append(-100)
            elif word_idx != previous_word_idx:
                # First subword of the word → real label
                label_ids.append(label2id[sentence_labels[word_idx]])
            else:
                # Remaining subwords → ignore in loss
                label_ids.append(-100)

            previous_word_idx = word_idx

        all_label_ids.append(label_ids)

    encodings["labels"] = torch.tensor(all_label_ids)
    return encodings
```

```
train_encodings = tokenize_and_align_labels(train_tokens, train_labels, tokenizer, label2id)
test_encodings  = tokenize_and_align_labels(test_tokens,  test_labels,  tokenizer, label2id)

for k, v in train_encodings.items():
    print(k, v.shape)
```

```
input_ids torch.Size([2578, 86])
token_type_ids torch.Size([2578, 86])
attention_mask torch.Size([2578, 86])
labels torch.Size([2578, 86])
```

In [52]:

```
from torch.utils.data import Dataset

class NERDataset(Dataset):
    def __init__(self, encodings):
        self.encodings = encodings

    def __len__(self):
        return self.encodings["input_ids"].shape[0]

    def __getitem__(self, idx):
        return {key: val[idx] for key, val in self.encodings.items()}

train_dataset = NERDataset(train_encodings)
test_dataset  = NERDataset(test_encodings)

len(train_dataset), len(test_dataset)
```

Out[52]:

```
(2578, 4712)
```

3.Data preprocessing and tokenization

1. Preparing the data in the correct format

2. Tokenizing the data correctly

In [53]:

```
def load_conll(path):
    sentences = []
    sentence = []

    with open(path, "r", encoding="utf-8") as f:
        for line in f:
            line = line.strip()

            # Sentence boundary or metadata
            if not line or line.startswith("#"):
                if sentence:
                    sentences.append(sentence)
                    sentence = []
                continue

            parts = line.split()
            token, label = parts[0], parts[-1]
            sentence.append((token, label))

    if sentence:
        sentences.append(sentence)

    return sentences


train_path = "/content/wnut 16.txt.conll"
test_path  = "/content/wnut 16test.txt.conll"

train_sentences = load_conll(train_path)
test_sentences  = load_conll(test_path)

print("Train sentences:", len(train_sentences))
print("Example sentence:", train_sentences[0][:8])
```

```
Train sentences: 2578
Example sentence: [('@SammieLynnsMom', 'O'), ('@tg10781', 'O'), ('they', 'O'), ('will', 'O'), ('be',
'O'), ('all', 'O'), ('done', 'O'), ('by', 'O')]
```

In [54]:
```
train_tokens = [[tok for tok, lab in sent] for sent in train_sentences]
train_labels = [[lab for tok, lab in sent] for sent in train_sentences]

test_tokens  = [[tok for tok, lab in sent] for sent in test_sentences]
test_labels  = [[lab for tok, lab in sent] for sent in test_sentences]
```

In [55]:
```
encodings = tokenizer(
    train_tokens,
    is_split_into_words=True,   # VERY IMPORTANT
    truncation=True,
    padding=True,
    max_length=128
)
```

In [56]:
```
import torch

label_ids = []

for i, sentence_labels in enumerate(train_labels):
    word_ids = encodings.word_ids(batch_index=i)
    previous_word_idx = None
    sentence_label_ids = []

    for word_idx in word_ids:
        if word_idx is None:
            sentence_label_ids.append(-100)  # special tokens
        elif word_idx != previous_word_idx:
            sentence_label_ids.append(label2id[sentence_labels[word_idx]])
        else:
            sentence_label_ids.append(-100)  # subword tokens

        previous_word_idx = word_idx

    label_ids.append(sentence_label_ids)

label_ids = torch.tensor(label_ids)
```

In [57]:
```
train_encodings = encodings
train_encodings["labels"] = label_idstest_encodings = tokenizer(
    test_tokens,
    is_split_into_words=True,
    truncation=True,
    padding=True,
    max_length=128
)

test_label_ids = []

for i, sentence_labels in enumerate(test_labels):
    word_ids = test_encodings.word_ids(batch_index=i)
    previous_word_idx = None
    sentence_label_ids = []

    for word_idx in word_ids:
        if word_idx is None:
            sentence_label_ids.append(-100)
        elif word_idx != previous_word_idx:
            sentence_label_ids.append(label2id[sentence_labels[word_idx]])
        else:
            sentence_label_ids.append(-100)

        previous_word_idx = word_idx

    test_label_ids.append(sentence_label_ids)

test_encodings["labels"] = torch.tensor(test_label_ids)
```

After converting the CoNLL data into sentence-level token and label sequences, we tokenize the data using a pretrained BERT tokenizer. As BERT performs subword tokenization, labels are aligned such that each word-level label

is assigned to the first corresponding subword token, while the remaining subword tokens and special tokens are masked using −100 to avoid contributing to the training loss.

4.Train the model

In [58]:
```
from transformers import AutoTokenizer, AutoModelForTokenClassification, TrainingArguments, Trainer

training_args = TrainingArguments(
    output_dir="./ner_results",
    eval_strategy="epoch",
    save_strategy="epoch",
    learning_rate=5e-5,
    per_device_train_batch_size=8,
    per_device_eval_batch_size=8,
    num_train_epochs=1,
    weight_decay=0.01,
    logging_dir="./logs",
    logging_steps=100,
    save_total_limit=2,
    report_to="none"
)
```

In [59]:
```
import numpy as np
from seqeval.metrics import classification_report, f1_score
```

In [60]:
```
def compute_metrics(p):
    predictions, labels = p
    predictions = np.argmax(predictions, axis=2)

    true_predictions = []
    true_labels = []

    for pred, lab in zip(predictions, labels):
        curr_preds = []
        curr_labels = []
        for p_i, l_i in zip(pred, lab):
            if l_i != -100:
                curr_preds.append(id2label[p_i])
                curr_labels.append(id2label[l_i])
        true_predictions.append(curr_preds)
        true_labels.append(curr_labels)

    return {
        "f1": f1_score(true_labels, true_predictions)
    }
```

In [61]:
```
from torch.utils.data import Subset

# use only first 20 examples for demo
small_train_indices = list(range(20))
small_train_dataset = Subset(train_dataset, small_train_indices)
```

In [62]:
```
from transformers import TrainingArguments, Trainer

training_args = TrainingArguments(
    output_dir="./ner_results_small",
    max_steps=10,
    per_device_train_batch_size=4,
    per_device_eval_batch_size=8,
    learning_rate=5e-5,
    logging_steps=5,
    report_to="none"
)
```

In [63]:
```
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=small_train_dataset,
    eval_dataset=test_dataset,   # ok to keep full test
    tokenizer=tokenizer,
    compute_metrics=compute_metrics,
```

```
)

trainer.train()
```

[10/10 00:53, Epoch 2/2]

Step	Training Loss
5	2.117100
10	0.545800

Out[63]:
```
TrainOutput(global_step=10, training_loss=1.3314575910568238, metrics={'train_runtime': 59.1615, 'train
_samples_per_second': 0.676, 'train_steps_per_second': 0.169, 'total_flos': 1755889155360.0, 'train_los
s': 1.3314575910568238, 'epoch': 2.0})
```

5. Try different hyperparameters

In [64]:
```
from transformers import AutoModelForTokenClassification, TrainingArguments, Trainer

model_hp = AutoModelForTokenClassification.from_pretrained(
    model_name,
    num_labels=len(unique_labels),
    label2id=label2id,
    id2label=id2label,
)
```

```
Some weights of BertForTokenClassification were not initialized from the model checkpoint at bert-base-ca
sed and are newly initialized: ['classifier.bias', 'classifier.weight']
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and infer
ence.
```

In [65]:
```
training_args_hp = TrainingArguments(
    output_dir="./ner_results_hp",
    learning_rate=3e-5,
    per_device_train_batch_size=8,
    per_device_eval_batch_size=8,
    num_train_epochs=1,
    weight_decay=0.01,
    logging_steps=100,
)
```

In [66]:
```
from torch.utils.data import Subset

small_train_indices = list(range(20))  # 20 examples is enough to "demonstrate"
small_train_dataset = Subset(train_dataset, small_train_indices)
```

In [67]:
```
from transformers import TrainingArguments, Trainer

training_args_hp = TrainingArguments(
    output_dir="./ner_results_hp",
    max_steps=10,
    per_device_train_batch_size=4,
    per_device_eval_batch_size=8,
    learning_rate=3e-5,
    weight_decay=0.05,
    logging_steps=5,
    report_to="none"
)
```

In [68]:
```
model_hp = AutoModelForTokenClassification.from_pretrained(
    model_name,
    num_labels=len(unique_labels),
    label2id=label2id,
    id2label=id2label,
)
```

```
Some weights of BertForTokenClassification were not initialized from the model checkpoint at bert-base-ca
sed and are newly initialized: ['classifier.bias', 'classifier.weight']
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and infer
ence.
```

In [69]:
```
trainer_hp = Trainer(
    model=model_hp,
    args=training_args_hp,
    train_dataset=small_train_dataset,
    eval_dataset=test_dataset,
    tokenizer=tokenizer,
    compute_metrics=compute_metrics,
)

trainer_hp.train()
```

[10/10 01:46, Epoch 2/2]

Step	Training Loss
5	2.337100
10	1.283100

Out[69]:
```
TrainOutput(global_step=10, training_loss=1.810091257095337, metrics={'train_runtime': 111.3367, 'train_samples_per_second': 0.359, 'train_steps_per_second': 0.09, 'total_flos': 1755889155360.0, 'train_loss': 1.810091257095337, 'epoch': 2.0})
```

6. Make predictions

1. Join the subtokens

2. Prediction on your own sentences

In [70]:
```
import numpy as np

predictions, labels, _ = trainer.predict(test_dataset)
predictions = np.argmax(predictions, axis=2)
```

In [71]:
```
true_predictions = []
true_labels = []

for pred, lab in zip(predictions, labels):
    sent_preds = []
    sent_labels = []
    for p_i, l_i in zip(pred, lab):
        if l_i != -100:  # ignore special/subword positions
            sent_preds.append(id2label[p_i])
            sent_labels.append(id2label[l_i])
    true_predictions.append(sent_preds)
    true_labels.append(sent_labels)
```

In [72]:
```
!pip install -q seqeval
from seqeval.metrics import classification_report, f1_score

print(classification_report(true_labels, true_predictions))
print("F1 score:", f1_score(true_labels, true_predictions))
```

```
              precision    recall  f1-score   support

     company       0.00      0.00      0.00       586
    facility       0.00      0.00      0.00       244
     geo-loc       0.00      0.00      0.00       768
       movie       0.00      0.00      0.00        28
musicartist       0.00      0.00      0.00       180
       other       0.00      0.00      0.00       536
      person       0.00      0.00      0.00       466
     product       0.00      0.00      0.00       216
  sportsteam       0.00      0.00      0.00       128
      tvshow       0.00      0.00      0.00        24

   micro avg       0.00      0.00      0.00      3176
   macro avg       0.00      0.00      0.00      3176
weighted avg       0.00      0.00      0.00      3176

F1 score: 0.0
```

In [73]:
```
import torch

def predict_sentence(sentence, tokenizer, model):
    # simple whitespace tokenization for your own sentence
    words = sentence.split()

    enc = tokenizer(
        words,
        is_split_into_words=True,
        return_tensors="pt",
        truncation=True,
        padding=True,
    )

    with torch.no_grad():
        outputs = model(**enc)

    preds = torch.argmax(outputs.logits, dim=2)[0].numpy()
    word_ids = enc.word_ids()

    final_labels = []
    prev_word_id = None

    for p_id, w_id in zip(preds, word_ids):
        # skip special tokens and subword continuations
        if w_id is None or w_id == prev_word_id:
            continue
        final_labels.append(id2label[p_id])
        prev_word_id = w_id

    return list(zip(words, final_labels))
```

In [74]:
```
my_sentence = "Elon Musk visited London and met Google executives"
predict_sentence(my_sentence, tokenizer, model)
```

Out[74]:
```
[('Elon', 'O'),
 ('Musk', 'O'),
 ('visited', 'O'),
 ('London', 'O'),
 ('and', 'O'),
 ('met', 'O'),
 ('Google', 'O'),
 ('executives', 'O')]
```

In [75]:
```
def show_subtokens(sentence, tokenizer):
    enc = tokenizer(sentence, return_tensors="pt")
    subtokens = tokenizer.convert_ids_to_tokens(enc["input_ids"][0])
    print(subtokens)

show_subtokens("NewYorkCity", tokenizer)
```

```
['[CLS]', 'New', '##Y', '##or', '##k', '##C', '##ity', '[SEP]']
```

Questions

1. Defining the problem statements and where can this and modifications of this be used?

Answer :

The objective is to build a Named Entity Recognition (NER) system that automatically identifies and classifies named entities (such as persons, locations, organizations, products, etc.) in Twitter text without relying on user-provided hashtags.

Applications and modifications:

- Trend detection: Identify emerging entities in real-time tweets.
- Improved search & recommendations: Entity-aware search on social media.
- Targeted advertising: Linking brands/products mentioned in tweets.
- Content moderation: Detect sensitive entities or misinformation.

- Customer support & sentiment analysis: Entity-based sentiment tracking.
- Domain adaptation (modification): Extending NER to medical, legal, or financial tweets using domain-specific labels.

2. Explain the data format (conll bio format)

Answer:

- In CoNLL BIO format, the dataset is organized line by line:
- Each line contains one token and its corresponding label
- Sentences are separated by blank lines
- Labels follow the BIO scheme:
  - B - Beginning of an entity
  - I - Inside an entity
  - O - Outside any entity

3. What other ner data annotation formats are available and how are they different

Answer:

Common annotation formats include:

- IO / BIOES / BILOU:
  - BIOES adds End (E) and Single (S) tags for better boundary detection
  - BILOU (Begin, Inside, Last, Outside, Unit) is widely used in spaCy
- Span-based annotation:
  - Entities stored as character start–end indices in raw text (used in JSON)
- Inline/XML tagging:
  - Entities embedded directly in text (e.g., Harry)

Key difference:

BIO-style formats are token-based, whereas span-based formats are character-based and easier to use with raw text.

4. Why do we need tokenization of the data in our case

Answer:

Tokenization is required because:

- Transformers operate on tokens, not raw text
- BERT uses WordPiece subword tokenization, which splits rare words into smaller units
- Proper tokenization allows:
  - Handling of unknown or misspelled Twitter words
  - Efficient vocabulary usage
  - Alignment between tokens and labels for NER

Without tokenization, the model cannot process variable-length text sequences.

5. What other models can you use for this task

Answer:

- Other models suitable for NER include:
- Traditional models:
  - Conditional Random Fields (CRF)
  - Hidden Markov Models (HMM)
- Deep learning models:
  - BiLSTM
  - BiLSTM + CRF
- Transformer-based models:
  - RoBERTa
  - DistilBERT
  - ALBERT
  - ELECTRA
  - BERTweet (specifically trained on Twitter data)

Transformer-based models generally outperform traditional approaches due to better contextual understanding.

6. Did early stopping have any effect on the training and results.

Answer:

Yes, early stopping helps:

- Prevent overfitting by halting training when validation performance stops improving
- Reduce training time and compute cost

In practice, early stopping may slightly reduce training loss but often improves generalization and stabilizes evaluation metrics such as F1-score.

7. How does the BERT model expect a pair of sentences to be processed?

Answer:

- Segment embeddings distinguish Sentence A and Sentence B
- Attention mechanism allows interaction across both sentences
- Commonly used for:
  - Question answering
  - Natural language inference
  - Sentence similarity tasks

8. Why choose Attention based models over Recurrent based ones?

Answer:

Attention-based models (Transformers) are preferred because:

- They capture long-range dependencies better than RNNs
- Computation is parallelizable, leading to faster training
- They avoid vanishing gradient issues
- Self-attention focuses on relevant words regardless of distance

Recurrent models process tokens sequentially, making them slower and less scalable.

9. Differentiate BERT and simple transformers

Answer:

BERT differs from simple Transformer architectures by using bidirectional contextual pretraining, which allows each token to attend to both its left and right context simultaneously. This makes BERT particularly effective for understanding tasks such as NER, where the meaning of a word strongly depends on surrounding tokens.

In [75]:

```
```